

CUDA Optimized Libraries and Kernel Performance: Interview Guide

Optimized CUDA libraries package years of architecture-specific tuning behind stable APIs. A strong GPU programmer does not start by writing every primitive from scratch: first identify the mathematical operation, choose the highest-level library that expresses it, then profile the whole application. Library calls still require correct layouts, streams, workspace, precision, and problem sizes to perform well.

Library selection map

Need	First choice
Dense vectors/matrices, GEMM	cuBLAS / cuBLASLt
Neural-network operators and fused graphs	cuDNN
Fourier transforms	cuFFT
Sparse linear algebra	cuSPARSE
STL-like parallel C++ algorithms	Thrust
Low-level reusable CUDA collectives	CUB
Multi-GPU collective communication	NCCL

cuBLAS

1. Overview

cuBLAS is NVIDIA's GPU-accelerated implementation of BLAS, the standard Basic Linear Algebra Subprograms interface. It supplies tuned vector, matrix-vector, and matrix-matrix operations. The best-known operation is GEMM:

$$C \leftarrow \alpha \, op(A)op(B) + \beta C$$

It matters because matrix multiplication dominates scientific computing, recommendation systems, transformers, graphics, and many numerical applications. NVIDIA tunes implementations for memory hierarchy, instruction scheduling, Tensor Cores, data types, and each GPU generation. Interviewers ask about cuBLAS to see whether a candidate recognizes standard operations, understands layout and precision, and knows when a library beats a handwritten kernel.

2. Core Idea

Imagine multiplying large matrices as moving boxes through a factory. A naive worker fetches every item from a distant warehouse repeatedly. cuBLAS divides matrices into tiles, stages reused values in registers/shared memory, overlaps movement with arithmetic, and assigns tiles to hardware efficiently.

```

cublasHandle_t h;
cublasCreate(&h);
float alpha = 1.0f, beta = 0.0f;
cublasSgemm(h, CUBLAS_OP_N, CUBLAS_OP_N,
            m, n, k, &alpha,
            A, m, B, k, &beta, C, m);
cublasDestroy(h);

```

Step by step: create a reusable handle; describe transpose modes and dimensions; provide device pointers and leading dimensions; cuBLAS selects an implementation; work is enqueued on the handle's CUDA stream; synchronization happens only when required by stream dependencies or host access. The classic API assumes column-major BLAS conventions. A row-major multiplication can be expressed by swapping operands/order or by using cuBLASLt layouts—never silently pass row-major buffers as column-major.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
BLAS levels	Level 1 is vector-vector, Level 2 matrix-vector, Level 3 matrix-matrix. Arithmetic intensity generally rises from Level 1 to 3.	AXPY, GEMV, GEMM	Why GEMM uses compute better than GEMV.
Handles and streams	A handle stores library context and has an associated stream. Reuse it; do not create one per operation.	<code>cublasSetStream(h, s)</code>	Asynchronous execution and thread-safety expectations.
Column-major layout	Classic BLAS indexes column-major matrices using a leading dimension.	<code>A(row,col)=A[row+col*lda]</code>	Diagnose transposed/wrong results.
Leading dimension	Physical stride between consecutive columns; it need not equal logical row count for submatrices/padding.	<code>lda >= max(1,m)</code> for non-transposed A	Leading dimension is not always matrix width.
GEMM batching	Strided-batched or pointer-array APIs amortize launches for many small independent products.	Attention heads, small systems	One large launch versus thousands of tiny calls.

Subtopic	Meaning and why it matters	Example	Common interview angle
Mixed precision	Input, accumulation, and output types can differ. Tensor Cores favor FP16/BF16/TF32/FP8 paths depending on hardware/API.	FP16 inputs, FP32 accumulation	Speed, range, accuracy, and numerical validation.
cuBLASLt	Flexible matmul API with explicit layouts, heuristics, workspace, algorithm selection, and fused epilogues.	GEMM + bias + GELU	When to prefer it over classic <code>cublasGemmEx</code> .
Tensor Cores	Specialized matrix multiply-accumulate units; dimensions, alignment, type, math mode, and layout affect eligibility.	Transformer projection	Tensor Core capable does not mean every GEMM uses them efficiently.
Workspace/algorithms	Some fast algorithms need scratch memory. More workspace can expand choices.	cuBLASLt preference	Performance versus memory budget.
Numerical behavior	Parallel reductions reorder floating-point additions; fast modes may change precision.	TF32 GEMM differs from strict FP32	Tolerance-based testing and reproducibility.

4. Real-World Example

A recommendation backend computes scores `users x item_embeddings^T`. A handwritten dot-product kernel may work, but the operation is GEMM. The server packs many users into a batch, keeps matrices on device, calls cuBLASLt, and optionally fuses bias/activation in the epilogue. Large batches raise arithmetic intensity and amortize launch overhead. For a single user, GEMV or a batched strategy may be more appropriate; calling a huge GEMM interface does not make a tiny problem large.

5. Diagrams / Mental Models

Host description	GPU implementation
A, B, C pointers	
dimensions/strides	└─> cuBLAS heuristic -> tiled loads -> MMA -> epilogue -> C

types/transpose	}	reuse A/B	alpha,beta/bias
stream/workspace			

BLAS level	Representative operation	Work	Data traffic tendency	Typical bound
1	AXPY $y=ax+y$	$O(n)$	$O(n)$	Memory bandwidth
2	GEMV $y=Ax$	$O(mn)$	$O(mn)$	Usually bandwidth
3	GEMM $C=AB$	$O(mnk)$	$O(mn+mk+nk)$	Often compute for large matrices

6. Common Interview Questions

- What is cuBLAS?** A tuned GPU implementation of standard dense linear-algebra routines. Expected: BLAS levels and GPU acceleration. Mistake: calling it a general deep-learning library.
- Why use cuBLAS instead of a custom matrix-multiply kernel?** It contains architecture-specific tiling, vectorization, Tensor Core paths, heuristics, and edge handling. Expected: faster development and usually better performance. Mistake: claiming it is always fastest for fused or unusual operations.
- What does GEMM compute?** $C=\alpha*\text{op}(A)*\text{op}(B)+\beta*C$. Expected: transpose options and scalar accumulation. Mistake: omitting existing C when β is nonzero.
- What is a leading dimension?** The physical stride between columns in column-major storage. Expected: padding/submatrix support. Mistake: treating it as universally equal to logical columns.
- How do you use row-major matrices?** Transform the operation by swapping operands/transposes or describe row-major layouts with cuBLASLt. Expected: mathematical verification. Mistake: just swapping m and n .
- Why is GEMM usually faster per FLOP than GEMV?** GEMM reuses matrix tiles and has high arithmetic intensity; GEMV rereads much data for few operations. Mistake: attributing it only to more threads.
- When does batched GEMM help?** Many independent small matrices with compatible shapes can share launch/dispatch overhead. Mistake: launching one cuBLAS call per matrix.
- What is cuBLASLt?** A flexible matmul-oriented API with explicit layouts, algorithm heuristics, workspace, and epilogue fusion. Mistake: saying it replaces all BLAS operations.
- How can Tensor Cores change accuracy?** Inputs or products may use reduced precision while accumulation may be wider; results can differ from strict FP32. Mistake: equating FP32 output with FP32 multiplication.
- Are cuBLAS calls synchronous?** GPU work is normally enqueued asynchronously on the configured stream; scalar pointer mode and later synchronization affect host interaction. Mistake: timing only the host call without stream synchronization/events.

7. Deep-Dive Questions

- Why can a cuBLAS GEMM be slow?** Tiny or skinny shapes underutilize the GPU; poor alignment/layout can exclude fast kernels; repeated handle creation, transfers, default-stream serialization, insufficient workspace, or unsupported precision paths add overhead. Compare against size-specific expectations, not peak marketing FLOPS.
- Why might fusing an epilogue help?** Bias/activation performed inside matmul avoids writing and rereading the full output and removes launches. It helps when the fused pattern is supported and the

saved bandwidth dominates.

3. **How would you benchmark GEMM correctly?** Allocate once, warm up, use CUDA events on the same stream, run repetitions, synchronize the stop event, compute FLOPs as about $2mnk$, and validate results/tolerances. Exclude transfers only if the production design does too.
4. **What happens when beta is zero?** Mathematically old β contributes nothing; optimized implementations may avoid reading it. The pointer must still satisfy the API contract. Do not assume uninitialized memory is valid without documentation.
5. **Why can two algorithms give slightly different answers?** They tile and reduce in different orders; floating-point addition is non-associative, and mixed-precision modes alter rounding. Determinism and accuracy are separate requirements from mathematical correctness.

8. Comparison Tables

Classic cuBLAS	cuBLASLt
Broad BLAS API	Matmul-focused flexible API
Traditional column-major conventions	Explicit matrix layout descriptors
Simpler GEMM call	More setup and heuristic control
Limited epilogue fusion	Bias/activation and other supported epilogues
Good default for standard BLAS	Best when layout, fusion, or tuning control matters
Custom GEMM	cuBLAS
Can fuse application-specific work	Highly tuned standard operation
Full control	Lower maintenance
Educational/specialized value	Best first production choice
Must retune across GPUs/shapes	Vendor tracks architectures

9. Common Mistakes

- Recreating/destroying handles inside the hot loop.
- Passing row-major data with column-major dimensions and getting a plausible but wrong transpose.
- Timing asynchronous calls with CPU clocks and no synchronization.
- Using many tiny GEMMs instead of batching or fusion.
- Assuming peak Tensor Core throughput applies to every shape.
- Ignoring α , β , compute type, workspace, alignment, or leading dimensions.
- Comparing results bit-for-bit across algorithms without a numerical requirement.

10. Edge Cases / Special Cases

Zero-sized dimensions, nonunit vector strides, aliasing, submatrices, odd dimensions, transposed operands, complex conjugate transpose, very skinny matrices, and negative increments can change legal parameters or performance. A handle's stream and pointer mode are mutable state, so sharing one carelessly across host threads is risky. Small matrices are often launch-bound. Unsupported layouts or insufficient workspace may select a slower fallback rather than fail.

11. How to Explain in Interview

"cuBLAS is NVIDIA's tuned dense linear-algebra library. I use it first for BLAS operations—especially GEMM—because it handles tiling, memory reuse, Tensor Cores, and architecture-specific algorithms. I still choose correct layouts, leading dimensions, streams, precision, batching, and workspace, then profile; for flexible matmul layouts or fused epilogues I consider cuBLASLt."

12. Quick Revision Notes

- GEMM: $C = \alpha * op(A) * op(B) + \beta * C$.
- BLAS 1 tends to bandwidth-bound; BLAS 3 can be compute-bound.
- Classic API is column-major by convention; `lda` is a stride.
- Reuse handles, set streams intentionally, batch small operations.
- Tensor Core use depends on type, shape, alignment, and selected algorithm.
- Interview trap: a host call returning does not mean GPU work finished.

13. Practice Tasks

1. Multiply two matrices with naive CUDA, tiled CUDA, and `cublasSgemm`; validate and plot throughput by size.
2. Store the same data row-major and column-major and derive correct calls on paper.
3. Compare 1,000 small GEMMs with a loop versus strided-batched GEMM.
4. Benchmark FP32 and an allowed mixed-precision mode; report speed and numerical error.
5. Use Nsight Systems to verify transfers, GEMM, and downstream kernels overlap as intended.

14. Final Cheat Sheet

Item	Remember
Core definition	Tuned GPU BLAS for dense linear algebra
Why it matters	Standard, portable API to highly optimized kernels
Most asked	GEMM equation, layout, leading dimensions, Tensor Cores
Common comparison	Classic cuBLAS versus cuBLASLt
One-line answer	Use cuBLAS for standard dense math; custom code must justify what the library cannot express.

cuDNN

1. Overview

cuDNN is NVIDIA's GPU-accelerated library of primitives and operation graphs for deep neural networks. It implements convolution/cross-correlation, matmul, attention, normalization, pooling, softmax, pointwise operations, and supported fusions. Frameworks such as PyTorch and TensorFlow commonly dispatch eligible GPU work to cuDNN.

It matters because DNN operators have many algorithms whose performance depends on tensor shapes, data layout, precision, workspace, GPU architecture, and determinism constraints. Interviewers use cuDNN to test whether candidates understand that framework operators map to lower-level kernels and that algorithm selection/fusion are central to performance.

2. Core Idea

Think of cuDNN as a route planner. You describe the tensor journey—dimensions, strides, types, operators, and dependencies. Several routes may compute the same result: direct convolution, transformed methods, implicit matrix multiplication, or fused engines. cuDNN heuristics rank supported execution plans; an application may benchmark candidates and cache the winner.

```

Tensor descriptors + operation graph
      ↓ finalize
Candidate engine configurations
      ↓ filter/support/autotune
Execution plan + workspace
      ↓ execute on stream
Output tensors

```

A basic convolution consumes input x , filter w , optional bias, and convolution attributes (padding, stride, dilation, groups), producing y . Modern graph APIs can represent a sequence such as convolution $\rightarrow$ bias $\rightarrow$ ReLU, enabling fusion that avoids intermediate global-memory traffic.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Tensor descriptors	Dimensions, strides, data type, identity/alignment describe actual memory.	NCHW or NHWC image batch	Logical shape versus physical layout.
Convolution parameters	Padding, stride, dilation, groups, mode determine output and work.	3×3 stride-2 conv	Derive output dimensions.
Forward/backward	Training needs forward output, data gradient, and filter gradient.	dx, dw	Inference is not the whole workload.
Algorithm/engine choice	Multiple kernels implement one graph with different speed/workspace/numerics.	Heuristic mode then autotune	Why the fastest algorithm varies by shape.
Workspace	Scratch memory enables algorithms/transforms; budget constrains selection.	Execution-plan workspace	Memory-speed tradeoff.

Subtopic	Meaning and why it matters	Example	Common interview angle
Graph API and fusion	Express a DAG so supported patterns become fewer kernels.	conv+bias+ReLU	Fusion reduces traffic and launch overhead.
Layout	Channel-first/channel-last and vectorized layouts affect coalescing/Tensor Cores.	NCHW versus NHWC	Conversion cost can erase benefits.
Mixed precision	Storage and compute types differ; reduced precision increases throughput.	FP16/BF16 with FP32 accumulation	Loss scaling and numerical validation.
Determinism	Fast algorithms may use reductions with nondeterministic ordering.	Atomic accumulation in backward pass	Reproducible result versus fastest result.
Dynamic shapes	New dimensions can require plan selection/building; caching matters.	Variable sequence/batch lengths	Warm-up and plan-cache behavior.

4. Real-World Example

An image service runs a CNN block: convolution, bias, batch normalization, and activation. Launching separate kernels writes each intermediate tensor to global memory. A supported cuDNN operation graph can choose a fused engine, keeping intermediate values closer to execution units and performing fewer launches. The service builds and caches plans during warm-up, gives cuDNN a workspace budget, runs inference in a CUDA stream, and measures end-to-end latency including layout conversions.

5. Diagrams / Mental Models

```

Unfused: x -Conv-> t1 (DRAM) -Bias-> t2 (DRAM) -ReLU-> y
Fused:   x ----- [Conv + Bias + ReLU engine] -----> y
                fewer launches/intermediates

```

Convolution output for one spatial dimension:

$$out = \left\lfloor \frac{in + 2p - d(k - 1) - 1}{s} \right\rfloor + 1$$

where p is padding, d dilation, k kernel size, and s stride.

6. Common Interview Questions

1. **What is cuDNN?** A GPU library of optimized DNN primitives and operation graphs. Expected: convolution/attention/normalization/fusion. Mistake: saying it is a full training framework.

2. **How is cuDNN different from CUDA?** CUDA is the programming platform/runtime; cuDNN is a domain library built on it. Mistake: presenting them as alternatives.
3. **Why are there multiple convolution algorithms?** Shapes, layouts, precision, workspace, and hardware favor different implementations. Mistake: naming one universally best algorithm.
4. **What is workspace?** Temporary device storage used by an execution plan. Expected: speed-memory tradeoff and lifetime. Mistake: confusing it with model parameters.
5. **What does operator fusion save?** Intermediate global-memory reads/writes and launch overhead; it may enable cross-op optimization. Mistake: saying fusion always reduces arithmetic.
6. **What is a tensor descriptor?** Metadata describing dimensions, strides, type, and related properties; it does not own tensor data. Mistake: treating a descriptor as allocation.
7. **NCHW versus NHWC?** Two logical/physical axis orders; optimal choice depends on kernel, type, and hardware. Mistake: converting around every operator without counting cost.
8. **How is an engine selected?** Query heuristics, reject unsupported/configuration-incompatible choices, optionally benchmark candidates, then build/cache an execution plan. Mistake: assuming heuristics guarantee the measured fastest.
9. **What does deterministic execution mean?** Repeated runs under defined conditions produce reproducible results; restricting algorithms can reduce performance. Mistake: equating determinism with exact real-number accuracy.
10. **Why can the first inference be slow?** Context initialization, module loading/JIT, heuristic search, plan building, allocation, and cache warm-up. Mistake: using the first call as steady-state latency.

7. Deep-Dive Questions

1. **Why may a fused graph be slower?** Unsupported shapes can fall back; fusion may increase register pressure, reduce occupancy, or use a less specialized combined kernel. Benchmark the graph, including eliminated conversions and launches.
2. **How does convolution become matrix multiplication?** `im2col` conceptually lowers receptive-field patches into matrix columns and multiplies by flattened filters. Optimized implicit-GEMM kernels generate needed tiles on demand to avoid materializing the huge lowered matrix.
3. **How do you handle dynamic shapes?** Bucket common shapes, cache plans keyed by full graph/configuration, set a workspace policy, and measure cache misses. Unbounded recompilation/plan building can dominate latency.
4. **Why does NHWC often help reduced precision?** Contiguous channel data can match vectorized loads and Tensor Core-friendly kernels. It is a tendency, not a universal guarantee; the whole graph's layout matters.
5. **How would you debug incorrect cuDNN output?** Verify dimensions/strides/types, padding/stride/dilation/groups, convolution mode, alpha/beta, pointer alignment/lifetime, stream dependencies, and workspace; compare a small case against a CPU reference before blaming precision.

8. Comparison Tables

cuDNN	cuBLAS
DNN-domain operations/graphs	General dense linear algebra
Convolution, attention, normalization, fusion	GEMM, GEMV, vector operations
Tensor/operator semantics	Matrix/vector semantics

cuDNN	cuBLAS
May internally use GEMM-like methods	Does not understand neural-network layers
Legacy fixed-function style	Graph/frontend style
One predefined operation per call	Declarative DAG of tensor operations
Straightforward for isolated supported ops	Supports broader fusion and engine planning
Limited cross-operation visibility	Can optimize supported multi-op patterns

9. Common Mistakes

- Treating cuDNN as a neural-network framework or automatic model compiler.
- Ignoring tensor strides and assuming contiguous memory.
- Benchmarking only steady-state kernels while production rebuilds plans.
- Choosing the fastest plan without checking workspace or numerical notes.
- Forcing one layout per layer and paying repeated conversions.
- Expecting deterministic algorithms to be bit-identical across every version/device.
- Assuming reduced-precision storage determines accumulation precision.

10. Edge Cases / Special Cases

Grouped/depthwise convolution can behave very differently from dense convolution. Tiny batches may be launch- or latency-bound. Odd channel counts/alignment can miss vectorized paths. Empty tensors, large dilation, asymmetric padding, noncontiguous strides, and unsupported graph patterns require careful support checks. Training and inference use different graphs; backward algorithms can have larger workspace and different determinism behavior.

11. How to Explain in Interview

"cuDNN is NVIDIA's optimized DNN operator and graph library. I describe tensors and operations, let heuristics provide supported engine configurations, optionally autotune them, and cache an execution plan under a workspace and numerical policy. Its main advantages are tuned kernels, Tensor Core paths, and fusion, but layout conversions, dynamic-shape planning, and determinism constraints must be measured."

12. Quick Revision Notes

- Domain: DNN primitives and operation graphs, not model orchestration.
- Performance keys: shape, layout/strides, type, fusion, workspace, plan.
- Heuristic ranking is a starting point; autotuning measures actual hardware.
- First-call latency is not steady-state latency.
- Determinism, precision, and accuracy are related but distinct.
- Interview trap: a descriptor describes memory; it does not allocate it.

13. Practice Tasks

1. Derive convolution output shapes for varied padding, dilation, and stride.
2. Benchmark one convolution across NCHW/NHWC and multiple batch sizes.
3. Compare separate conv+bias+activation calls with an eligible fused graph.

4. Build a plan cache keyed by tensor shapes and measure warm/cold latency.
5. Compare FP32 and mixed-precision outputs against a high-precision reference.

14. Final Cheat Sheet

Item	Remember
Core definition	Tuned DNN primitives and fused operation graphs
Why it matters	Algorithm selection and fusion save development and runtime
Most asked	Layout, workspace, algorithms, fusion, determinism
Common comparison	cuDNN versus cuBLAS; graph versus fixed-function API
One-line answer	cuDNN turns tensor-operation descriptions into tuned GPU execution plans.

cuFFT

1. Overview

cuFFT is NVIDIA's GPU library for Fast Fourier Transforms. It converts signals between time/spatial and frequency domains and supports complex-to-complex (C2C), real-to-complex (R2C), complex-to-real (C2R), multiple dimensions, batches, strides, and advanced layouts.

FFTs power signal processing, spectral solvers, audio, radar, medical imaging, computational physics, and convolution of long signals. Interviewers ask about cuFFT to test algorithmic complexity, transform semantics, layout, plan reuse, normalization, and the boundary between a library call and an end-to-end pipeline.

2. Core Idea

A discrete Fourier transform computes how much of each frequency is present in a sequence. A direct DFT costs $O(N^2)$; FFT factorization reuses partial results to reach roughly $O(N \log N)$.

Analogy: instead of asking every attendee individually about every topic, repeatedly split the room into structured groups, combine group summaries, and reconstruct the same answer with far less work.

```

cufftHandle plan;
cufftPlan1d(&plan, N, CUFFT_C2C, batch);
cufftExecC2C(plan, d_in, d_out, CUFFT_FORWARD);
cufftExecC2C(plan, d_out, d_in, CUFFT_INVERSE);
// Scale d_in by 1/N in a separate/fused step if that is the desired convention.
cufftDestroy(plan);

```

Planning determines decomposition, kernels, workspace, and layout. Execution enqueues those kernels. Plans should be created outside hot loops and reused for matching configurations.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Transform types	C2C, R2C, and C2R have different input/output storage.	Real audio to frequency bins	Why R2C output has only $N/2+1$ complex bins.
Plans	Reusable description/resources for dimensions, batch, type, and layout.	One plan per recurring size	Plan creation versus execution cost.
Batch transforms	Many independent same-shaped transforms amortize launches and improve utilization.	FFT per sensor/channel	Prefer one batched plan to loops.
Advanced layout	<code>inembed</code> , <code>onembed</code> , stride, and distance describe padded/interleaved data.	Channels interleaved in one buffer	Avoid explicit transpose/copy when possible.
Workspace	Some plans require temporary device memory.	Auto-allocation or caller work area	Lifetime and concurrent execution.
In-place transforms	Input/output share storage but must obey documented padding/layout rules.	R2C padded real buffer	In-place is not just pointer equality.
Normalization	cuFFT transforms are generally unnormalized: <code>inverse(forward(x))</code> gives a scaled result.	Divide by N	Classic correctness interview trap.
Size factorization	Smooth sizes with small prime factors often run better than awkward sizes.	Pad 1009 to a suitable length if valid	Padding trades extra work for faster decomposition.
Hermitian symmetry	FFT of real data satisfies conjugate symmetry, so half-spectrum storage suffices.	$X[N-k] = \text{conj}(X[k])$	Interpret DC/Nyquist bins correctly.
Callbacks/fusion	Supported load/store callbacks can transform data during FFT I/O.	Windowing or scaling	Reduce extra memory passes, with constraints.

4. Real-World Example

An audio backend performs long convolution with an impulse response. It pads input blocks and the filter, computes their R2C FFTs, multiplies frequency bins pointwise, applies C2R, scales by transform length, and uses overlap-add/save to assemble the continuous result. The filter spectrum and plans are cached; input blocks use batched transforms. For short filters, direct convolution may be faster because FFT setup, padding, and extra passes dominate.

5. Diagrams / Mental Models

```
time signal -R2C FFT-> compact spectrum -multiply/filter-> spectrum -C2R FFT->
scaled time signal
      N real                      N/2+1 complex                      N real
```

```
FFT recursion (conceptual):
N samples
├ even-indexed N/2 FFT ┐
└ odd-indexed  N/2 FFT ┘ twiddle-factor combine -> N outputs
```

6. Common Interview Questions

1. **What does cuFFT provide?** GPU implementations of multidimensional and batched FFTs for real/complex data. Mistake: saying it performs arbitrary filtering automatically.
2. **FFT versus DFT complexity?** DFT is $O(N^2)$; FFT computes the same transform in $O(N \log N)$. Mistake: describing FFT as an approximation.
3. **What is a cuFFT plan?** A reusable transform configuration holding algorithm/resource decisions. Mistake: creating it for every execution.
4. **Does inverse FFT restore the original input directly?** Usually it returns N times the original for a length- N forward/inverse pair, so scale according to convention. Mistake: forgetting normalization.
5. **Why does R2C store $N/2+1$ complex values?** Real inputs create a Hermitian-symmetric spectrum; the remaining bins are redundant. Mistake: allocating only $N/2$ bins.
6. **When should transforms be batched?** When many independent transforms share shape/type/layout. Expected: amortized launch and better utilization. Mistake: a host loop of single FFTs.
7. **What is an in-place FFT risk?** Required padded layout differs, particularly for R2C/C2R, and overwritten input cannot be reused. Mistake: merely passing the same pointer.
8. **Why are some sizes faster?** FFT decompositions favor factorizations supported by efficient radix kernels; large prime factors may require costlier algorithms. Mistake: assuming powers of two are the only efficient sizes.
9. **Are plan creation and execution asynchronous?** Execution is enqueued on the plan's stream; plan creation can perform host work/allocation and should not be included unintentionally in steady-state timing. Mistake: timing the API return only.
10. **When is FFT convolution better than direct convolution?** Usually for sufficiently long kernels/signals or reusable filter spectra; crossover depends on size, batching, hardware, and transfers. Mistake: using asymptotics alone for small inputs.

7. Deep-Dive Questions

1. **How do you process data larger than GPU memory?** Stream chunks with overlap-save/add for convolution or use a distributed/out-of-core decomposition; count PCIe/network traffic and boundary overlap. A single enormous logical transform is harder than independent chunk FFTs.
2. **Why can padding improve performance?** A nearby smooth length may use efficient radices enough to offset computing extra samples. Padding is valid only if it preserves the mathematical operation, such as zero-padding linear convolution.

3. **How do strides affect speed?** Non-unit or poorly aligned strides cause inefficient global accesses and may require internal transposes. Sometimes an explicit, reusable layout conversion is faster; measure the entire pipeline.
4. **Can one plan execute concurrently?** Stream association and workspace sharing govern safety. Independent concurrent executions commonly need distinct plans/work areas or explicit ordering; follow the version's documented contract.
5. **How do you validate a spectral pipeline?** Test impulses, constant signals, single sinusoids, and random inputs; check DC/Nyquist handling, conjugate symmetry, sign convention, scaling, padding, and error tolerance.

8. Comparison Tables

Direct convolution	FFT-based convolution
$O(NK)$	Roughly $O(L \log L)$ with padded length L
Low setup; good for short kernels	FFT plans, padding, pointwise multiply
Easy streaming boundaries	Requires overlap-add/save
Often better at small K	Often better for long reusable filters

C2C	R2C/C2R
Full complex input/output	Exploits real-signal symmetry
N complex outputs	$N/2+1$ complex values for length N real input
Simpler symmetric layout	In-place padding rules need care

9. Common Mistakes

- Creating plans in the processing loop.
- Forgetting inverse scaling or applying it twice.
- Using incorrect R2C output allocation/in-place padding.
- Confusing transform direction/sign convention.
- Benchmarking a single tiny FFT and expecting full GPU utilization.
- Ignoring plan workspace and concurrent-use lifetime.
- Performing costly layout copies without testing advanced strides/batching.

10. Edge Cases / Special Cases

Length 1, odd real-transform lengths, DC and Nyquist bins, large primes, multidimensional leading dimensions, integer overflow in very large sizes, overlapping in/out buffers, and noncontiguous batches require explicit care. C2R input must represent valid Hermitian data if a real output is expected. Some plan configurations may allocate significant workspace or trigger initialization on the first call.

11. How to Explain in Interview

"cuFFT computes optimized GPU Fourier transforms using reusable plans. I choose C2C or real transforms, specify dimensions, batch and layout, reuse the plan/workspace, execute on the intended stream, and

remember normalization. I use FFT-based processing when the size and reuse outweigh planning, padding, and memory traffic."

12. Quick Revision Notes

- FFT computes the DFT in $O(N \log N)$, not an approximation.
- R2C length N output: $N/2+1$ complex values.
- Forward+inverse generally needs $1/N$ scaling.
- Reuse plans; batch equal transforms.
- Smooth sizes often perform better, but benchmark.
- Interview traps: scaling, in-place padding, asynchronous timing.

13. Practice Tasks

1. Run C2C forward/inverse on an impulse and verify scaling.
2. Implement an R2C low-pass filter and reconstruct with C2R.
3. Compare batched FFTs against a loop of single transforms.
4. Plot runtime for nearby transform sizes with different factorizations.
5. Implement direct and overlap-save convolution and find the crossover size.

14. Final Cheat Sheet

Item	Remember
Core definition	Planned GPU FFT library
Why it matters	Fast spectral transforms for signal/scientific pipelines
Most asked	$O(N \log N)$, plans, batching, R2C layout, normalization
Common comparison	Direct versus FFT convolution
One-line answer	cuFFT is fastest when plans are reused and transform/layout overhead is amortized.

cuSPARSE

1. Overview

cuSPARSE is NVIDIA's GPU library for sparse linear algebra. Sparse matrices store mostly nonzero values plus indices rather than every zero. The library provides sparse matrix-vector/matrix multiplication, sparse triangular solves, format conversion, pruning and related routines through descriptor-based APIs.

It matters in graphs, scientific simulations, recommender systems, optimization, and sparse machine learning. Sparse performance is difficult because useful work and memory accesses are irregular. Interviewers ask about it to test storage-format reasoning, load balance, arithmetic intensity, preprocessing, and whether sparsity is actually beneficial.

2. Core Idea

For a dense matrix, every grid cell is stored. For a sparse matrix, store only occupied cells and enough addresses to locate them. It resembles a city map listing only buildings rather than every empty parcel: much smaller, but following addresses adds overhead and neighboring workers may receive very different amounts of work.

CSR example:

```
A = [10  0  0  2]
     [ 0  3  0  0]
     [ 4  0  5  0]

values = [10, 2, 3, 4, 5]
col_ind = [ 0, 3, 1, 0, 2]
row_ptr = [ 0, 2, 3, 5]
```

For SpMV $y=A*x$, each row traverses $[row_ptr[r], row_ptr[r+1])$, gathers elements of x using col_ind , multiplies, and reduces. cuSPARSE chooses parallel algorithms, but row-length imbalance and indirect reads remain properties of the data.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
CSR	Values and column indices grouped by row plus row offsets.	Efficient row-wise SpMV	Explain <code>row_ptr</code> length <code>rows+1</code> .
CSC	CSR-like storage by column.	Column access or transpose workflows	CSR of A resembles CSC of A^T conceptually.
COO	Store <code>(row,col,value)</code> triples; simple construction/conversion.	Edge list	More index traffic; duplicates may exist.
BSR/blocked formats	Store dense blocks and block indices.	Block-structured PDE matrix	Structure can improve locality/vectorization.
SELL/ELL-like formats	Regularize/pad row slices for balanced accesses.	Similar row lengths	Padding overhead versus regular execution.
SpMV/SpMM	Sparse $\times$ vector and sparse $\times$ dense matrix.	Iterative solver, GNN layer	SpMM has more reuse/arithmetic intensity.
Descriptors	Matrix/vector metadata: shape, indices, format, type, base.	CSR descriptor	Descriptor does not own buffers.

Subtopic	Meaning and why it matters	Example	Common interview angle
External workspace	Caller provides scratch sized by a buffer query.	<code>cusparseSpMV_bufferSize</code>	Reuse allocation across iterations.
Preprocessing	Analyze a stable sparsity pattern once to speed repeated calls.	Iterative solver	Amortization and active-buffer lifetime.
Determinism	Algorithm choices may trade bitwise reproducibility for speed.	Alternative SpMV algorithms	Atomics/reduction order and floating point.

4. Real-World Example

A conjugate-gradient solver repeatedly computes $q=A*p$ for the same sparse system matrix while vector values change. It stores **A** in CSR, creates descriptors once, queries/allocates workspace once, preprocesses the stable pattern when supported, and reuses everything for hundreds of SpMV calls. If the matrix is block-structured, BSR may reduce index overhead. End-to-end speed also depends on vector reductions and synchronization, so the solver profiles the full iteration rather than SpMV alone.

5. Diagrams / Mental Models

```

Sparse workflow
dense/COO input -> choose/convert format -> descriptor -> buffer query
                                     |
                                     +--> optional preprocess
                                     |
                                     v
                                repeated SpMV/SpMM calls

```

Matrix property	Likely starting format
General row-wise compute	CSR
Easy assembly / edge list	COO
Column-oriented operations	CSC
Dense fixed-size blocks	BSR
Similar row lengths, regular slices	SELL/ELL family

6. Common Interview Questions

- What is cuSPARSE?** A GPU library for sparse matrix representations and sparse linear-algebra operations. Mistake: calling it sparse memory allocation.
- Explain CSR.** Nonzeros by row, matching column indices, and `rows+1` offsets marking each row's range. Mistake: saying `row_ptr` stores row indices per value.
- Why can sparse operations be memory-bound?** Few FLOPs per loaded value/index, indirect vector gathers, and limited reuse. Mistake: assuming fewer multiplications always means proportionally faster.

4. **When is sparse storage worse than dense?** At moderate density, index/storage/control overhead and poor regularity can outweigh skipped zeros. Crossover depends on operation and hardware. Mistake: using a universal sparsity percentage.
5. **CSR versus COO?** CSR is compact and efficient for row operations; COO is simple to construct and stores a row index per nonzero. Mistake: ignoring duplicate/sorting semantics.
6. **What causes load imbalance in SpMV?** Rows have unequal nonzero counts, so work assigned by row finishes unevenly. Mistake: measuring only average nonzeros per row.
7. **Why preprocess SpMV?** Analyze a stable structure once and reuse acceleration data over repeated calls. Mistake: preprocessing a matrix used once.
8. **What is SpMM's advantage over SpMV?** Multiple dense RHS columns can reuse sparse values/indices and raise arithmetic intensity. Mistake: looping over many SpMVs without comparison.
9. **Do sparse indices need sorting?** Some modern operations allow unsorted indices, but sorting may affect performance and other routines' contracts; verify the chosen API. Mistake: assuming allowed means optimal.
10. **Why can sparse results be nondeterministic?** Parallel accumulation order/atomics can vary, changing floating-point rounding. Mistake: calling small rounding differences a data race automatically.

7. Deep-Dive Questions

1. **How would you choose a sparse format?** Measure density, row-length distribution, block structure, access direction, update frequency, conversion cost, index size, and target operation. Benchmark representative matrices; no format wins universally.
2. **Why can transpose SpMV be slower?** Natural CSR storage supports row-wise non-transpose access. Transpose may scatter/atomically combine or access irregularly; CSC or a precomputed transpose may pay off over repeated use.
3. **What if the pattern changes every iteration?** Descriptor updates, sorting/conversion, and preprocessing may dominate. Use a construction-friendly representation or batch structural updates; do not assume stable-pattern optimization applies.
4. **How does index width matter?** 64-bit indices support huge matrices but double index traffic versus 32-bit in many paths, harming a bandwidth-bound operation. Use the smallest safe supported type.
5. **How would you improve an imbalanced power-law graph SpMV?** Consider algorithms that split long rows across warps/blocks, reorder vertices, use COO/hybrid strategies, or switch representation. Include preprocessing cost and downstream locality in the decision.

8. Comparison Tables

Dense GEMV	Sparse SpMV	
Stores all values	Stores values plus indices	
Regular/coalesced access	Indirect and data-dependent access	
Predictable work per row	Row-length imbalance	
Better at high density	Better when zeros skipped outweigh metadata/irregularity	
CSR	COO	BSR
Row offsets + columns	Row + column per nonzero	Indices dense blocks

CSR	COO	BSR
Good general SpMV	Easy assembly/edge lists	Good block structure
Compact row metadata	Extra row-index traffic	Padding inside partial blocks

9. Common Mistakes

- Assuming sparsity guarantees speedup.
- Using the wrong index base or index width.
- Reallocating workspace/descriptors every iteration.
- Ignoring row-length distribution and only reporting density.
- Repeatedly converting formats without amortization.
- Treating descriptor metadata as owned data storage.
- Expecting bitwise identical results from every fast reduction algorithm.

10. Edge Cases / Special Cases

Empty rows, empty matrices, duplicate coordinates, explicit stored zeros, unsorted indices, one extremely long row, structural symmetry, integer-index overflow, and aliasing can change correctness or performance.

Operations differ in accepted formats/types/transposes. A preprocess buffer can be tied to a descriptor/configuration; mutating indices or reusing the buffer concurrently may invalidate assumptions even if values are allowed to change.

11. How to Explain in Interview

"cuSPARSE provides optimized sparse matrix formats and operations such as SpMV and SpMM. Sparse speed depends less on FLOP peak and more on index traffic, indirect accesses, and load balance. I choose a format from the matrix structure and operation, reuse descriptors/workspace, preprocess stable patterns, and compare against dense execution at the actual density."

12. Quick Revision Notes

- CSR: `values, col_ind, row_ptr[rows+1]`.
- Sparse operations are often bandwidth/latency/load-balance limited.
- Format choice is data- and operation-dependent.
- SpMM can reuse metadata better than repeated SpMV.
- Preprocess only pays with pattern reuse.
- Interview trap: fewer stored values does not automatically mean faster.

13. Practice Tasks

1. Convert a small dense matrix to COO and CSR by hand.
2. Implement CPU CSR SpMV and validate cuSPARSE output.
3. Benchmark CSR SpMV for uniform versus power-law row lengths.
4. Compare repeated SpMV with SpMM for multiple right-hand sides.
5. Measure preprocessing cost and determine the iteration count needed to amortize it.

14. Final Cheat Sheet

Item	Remember
Core definition	GPU sparse linear-algebra library
Why it matters	Skips zeros while managing sparse formats and irregular work
Most asked	CSR, format choice, load imbalance, SpMV versus SpMM
Common comparison	Sparse versus dense; CSR versus COO/BSR
One-line answer	cuSPARSE is valuable when saved zero work exceeds index and irregular-access overhead.

Thrust

1. Overview

Thrust is a C++ parallel algorithms library included with the CUDA Toolkit as part of CUDA Core Compute Libraries (CCCL). Its interface resembles the C++ Standard Library: containers, iterators, execution policies, and algorithms such as sort, reduce, scan, transform, copy, and partition. It can dispatch to GPU and CPU backends.

Thrust matters because much GPU data processing is composition of standard parallel patterns rather than a novel kernel. It is used in preprocessing, simulation pipelines, compaction, sorting, analytics, and rapid prototypes. Interviewers ask about it to test abstraction choice, iterator/policy semantics, synchronization, allocation overhead, and when higher-level code should give way to a fused kernel or CUB primitive.

2. Core Idea

Thrust separates **what** to do from **how** thousands of threads do it. Calling `transform` says “apply this operation to every item”; the backend chooses partitioning and launch details.

Analogy: instead of assigning each warehouse worker a route, give the manager a standard job—sort, filter, total—and let an optimized scheduling system distribute it.

```
thrust::device_vector<int> x{1, 2, 3, 4};
thrust::transform(x.begin(), x.end(), x.begin(),
    [] __device__ (int v) { return v * v; });
int sum = thrust::reduce(x.begin(), x.end(), 0); // 30
```

Step by step: `device_vector` owns device memory; device iterators select a CUDA backend (or an explicit policy does); `transform` launches parallel work; `reduce` combines values using an associative operation; returning a scalar to the host creates a synchronization/data-transfer boundary. Algorithm calls are composable, but each call may add a launch and intermediate memory pass.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Containers	<code>host_vector</code> and <code>device_vector</code> manage host/device storage.	Copy host input to device	Convenient ownership versus allocation/copy cost.
Algorithms	Parallel equivalents of STL patterns.	<code>sort</code> , <code>reduce</code> , <code>scan</code>	Recognize patterns before writing kernels.
Execution policies	Explicitly choose backend/stream behavior.	<code>thrust::cuda::par.on(stream)</code>	Never assume the intended stream accidentally.
Fancy iterators	Generate/transform/permutate views without materializing arrays.	counting, zip, transform iterators	Eliminate temporary storage and memory passes.
Functors/lambdaes	Element operation compiled for target system.	predicate for <code>copy_if</code>	Capture/device-callability constraints.
Scan	Prefix reduction: inclusive or exclusive.	offsets for compaction	Scan is not the same as scalar reduction.
Stable variants	Preserve equivalent-key input order, usually with a cost.	<code>stable_sort_by_key</code>	Use stability only when required.
Host results	Algorithms returning a host scalar need completion before return.	<code>reduce</code> result	Hidden synchronization in a pipeline.
Allocations	Temporary allocations and <code>device_vector</code> resizing can dominate small calls.	repeated sort in loop	Reuse buffers/allocators where measured.
Fusion	One <code>transform</code> can combine elementwise expressions; separate algorithms reread data.	transform-reduce	Express fused standard patterns first.

4. Real-World Example

A particle simulator removes inactive particles and calculates total energy. `copy_if` compacts active particles; `transform_reduce` computes energy without creating a temporary energy array. Counting/zip iterators combine indices and particle fields as views. The programmer uses one stream policy and preallocated buffers.

If profiling shows two remaining passes dominate, a custom fused kernel may compact and accumulate together—but only after handling its harder output allocation and reduction semantics.

5. Diagrams / Mental Models

```

Raw arrays -> iterators/views -> execution policy -> algorithm -> output
              |                   |
              |                   |
no-copy transformation      backend + stream
  
```

Pattern	Thrust algorithm
Map	<code>transform</code>
Filter/compact	<code>copy_if</code> , <code>remove_if</code>
Total/min/max	<code>reduce</code>
Prefix offsets	<code>inclusive_scan</code> , <code>exclusive_scan</code>
Map then total	<code>transform_reduce</code>
Grouped aggregation	<code>reduce_by_key</code>
Reorder	<code>sort</code> , <code>sort_by_key</code>

6. Common Interview Questions

1. **What is Thrust?** A high-level C++ parallel algorithms library with STL-like APIs and multiple backends. Mistake: calling it a CUDA runtime replacement.
2. **Why use it?** Standard operations become short, readable, optimized calls with less custom-kernel maintenance. Mistake: saying abstraction removes the need to profile.
3. **How is the backend selected?** By an explicit execution policy or iterator/system dispatch. Mistake: mixing host and device iterators casually.
4. **reduce versus scan?** Reduce returns one aggregate; scan returns every prefix aggregate. Mistake: saying scan is just a slower reduce.
5. **What is a fancy iterator?** A lazy view that generates or transforms values during access rather than storing another array. Mistake: assuming it allocates data.
6. **Why use transform_reduce?** It fuses mapping and reduction, avoiding an intermediate vector/pass. Mistake: materializing transformed data first without need.
7. **Is device_vector access from host cheap?** No; individual element access can cause a device transfer/synchronization. Bulk operations are preferred. Mistake: treating it like host RAM.
8. **Can Thrust run on a CUDA stream?** Yes, use an appropriate CUDA execution policy bound to the stream. Mistake: assuming all calls automatically use the application's stream.
9. **Why can several Thrust calls be slow?** Multiple launches, full memory passes, synchronization, and temporary allocation can dominate. Mistake: blaming templates/compile-time abstraction alone.
10. **When write a custom kernel?** When profiling proves missing fusion, specialized layout, communication, or semantics materially limit performance. Mistake: rewriting sort/scan from scratch as a first step.

7. Deep-Dive Questions

1. **Are reduction operators allowed to be non-associative?** Parallel grouping changes evaluation order, so correctness requires an associative operation in the algorithmic sense. Floating-point addition is only approximately associative, so expect rounding variation.
2. **How can iterators improve performance?** A transform/counting/zip view computes values at consumption, avoiding allocation, writes, reads, and often a launch. Complex iterator expressions can still affect compiler optimization, so measure.
3. **How do you reduce synchronization?** Keep results on device, bind operations to a stream, avoid host element accesses, compose/fuse algorithms, and synchronize only at real dependency boundaries.
4. **Thrust sort or custom radix sort?** Start with `sort/sort_by_key`; it already routes to optimized machinery. Drop lower only for known key constraints, storage control, fusion, or measured API overhead.
5. **What does performance portability mean here?** The algorithm interface can dispatch to CUDA or host systems, but equal semantics do not guarantee equal performance; data location and backend-specific policies still matter.

8. Comparison Tables

Thrust	CUB
High-level STL-like algorithms	Lower-level CUDA primitives
Iterator/container abstraction	Explicit warp/block/device collectives
Best for composing complete operations	Best inside custom kernels or for workspace/control
Less tuning surface	More CUDA-specific responsibility
Thrust algorithm	Custom kernel
Minimal code, maintained implementation	Maximum domain-specific fusion/control
Standard semantics	Custom correctness/edge handling
Possible intermediate passes	Can combine pipeline stages
First choice for standard patterns	Use after a measured limitation

9. Common Mistakes

- Reading/writing `device_vector[i]` repeatedly from host code.
- Mixing host and device iterator ranges.
- Omitting an explicit stream policy in a multistream application.
- Chaining elementwise calls when one transform expression suffices.
- Materializing data that a fancy iterator can generate.
- Assuming exact floating-point reduction order.
- Benchmarking allocation plus algorithm when production buffers are reusable—or excluding it when they are not.

10. Edge Cases / Special Cases

Empty ranges, overlapping input/output, stateful functors, invalidated iterators after resize, exceptions in device code, non-associative reductions, stability requirements, and mixed system iterators deserve attention. An algorithm returning a host value has different pipeline behavior from one writing a device result. A `device_vector` makes ownership easy, but raw pointers from it remain valid only while storage is not reallocated.

11. How to Explain in Interview

"Thrust is the high-level, STL-like CUDA parallel algorithms library. I use it for standard map, reduce, scan, sort, and compaction patterns, choose the execution policy and stream explicitly, and use fancy iterators or combined algorithms to avoid temporaries. I move to CUB or a custom kernel only when profiling shows missing fusion or control matters."

12. Quick Revision Notes

- High level: algorithms, containers, iterators, execution policies.
- `reduce` → scalar; `scan` → prefixes.
- Fancy iterators are lazy views, not allocations.
- `transform_reduce` avoids a temporary map result.
- Host scalar/element access can synchronize.
- Interview trap: concise source does not guarantee one GPU kernel.

13. Practice Tasks

1. Square and sum an array with `transform_reduce` and compare with two calls.
2. Compact positive values using `copy_if`.
3. Build histogram offsets using sort/reduce-by-key/scan.
4. Replace an intermediate index vector with a counting iterator.
5. Run a pipeline on a nondefault stream and verify its timeline.

14. Final Cheat Sheet

Item	Remember
Core definition	STL-like C++ parallel algorithms library
Why it matters	Express standard GPU patterns safely and quickly
Most asked	Policies, iterators, reduce versus scan, synchronization
Common comparison	Thrust versus CUB/custom kernels
One-line answer	Thrust describes the parallel pattern; the backend supplies the implementation.

CUB

1. Overview

CUB is a CUDA-specific library of reusable, tuned parallel primitives at thread, warp, block, and device scope. It is also part of CCCL and underlies many higher-level operations. It offers reductions, scans, sorting, selection, histograms, cooperative loads/stores, and related utilities.

CUB matters when a custom kernel needs a high-performance building block but writing a correct warp/block collective is unnecessary risk. It appears in database operators, renderers, graph processing, compilers, and frameworks. Interviewers ask about CUB to test synchronization scope, temporary storage, collective participation, hierarchy, and the distinction between an API-level algorithm and an in-kernel primitive.

2. Core Idea

CUB is a box of optimized gears. Thrust lets you request a complete machine operation; CUB lets you place a tuned reduction, scan, load, or sort gear inside your own machine.

```
template<int BLOCK_THREADS>
__global__ void block_sums(const float* x, float* sums, int n) {
    using Reduce = cub::BlockReduce<float, BLOCK_THREADS>;
    __shared__ typename Reduce::TempStorage temp;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float v = i < n ? x[i] : 0.0f;
    float total = Reduce(temp).Sum(v);
    if (threadIdx.x == 0) sums[blockIdx.x] = total;
}
```

Every thread in the block contributes a value, the collective cooperates through registers/shuffles/shared temporary storage, and one aggregate is produced. Template parameters specialize code for type/block size/algorithm and target architecture. Participation and synchronization rules are part of correctness.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Thread primitives	Per-thread load/store/reduction helpers.	vectorized thread load	Architecture specialization.
Warp collectives	Cooperation inside hardware/logical warps.	WarpReduce, WarpScan	Masks/participation and warp size.
Block collectives	Whole-block reduction, scan, load, sort, histogram.	BlockRadixSort	Shared TempStorage and block-wide participation.
Device algorithms	Complete grid-wide operations launched by the library.	DeviceReduce, DeviceScan	Two-step storage query pattern.
TempStorage	Correctly sized scratch type for a specialized collective.	shared memory instance	Reuse requires synchronization.

Subtopic	Meaning and why it matters	Example	Common interview angle
Logical warps	Compile-time subgroups within a hardware warp.	8-thread subgroup	Power-of-two behavior and grouping.
Items per thread	More items improve amortization/reuse but consume registers.	block load 4 keys/thread	Throughput versus occupancy/register pressure.
Algorithm variants	Select implementations with different resources/behavior.	block reduce algorithms	Defaults are good; tune only with evidence.
Cooperative I/O	Load/store patterns transpose between striped coalesced memory and blocked per-thread layout.	BlockLoad	Coalescing plus compute-friendly layout.
Stream/workspace reuse	Device algorithms accept streams and external scratch.	query once, allocate once	Cross-stream reuse hazards.

4. Real-World Example

A database filtering kernel evaluates a predicate for several records per thread. A block scan computes output offsets, one block count is reserved globally, and passing rows are scattered compactly. Using **cub::BlockScan** avoids implementing a fragile scan and leaves only domain-specific predicate and record movement in custom code. A later device-wide prefix operation can use **cub::DeviceScan**, with workspace allocated once.

5. Diagrams / Mental Models

```

Scope hierarchy
Device algorithm
├─ grid of blocks
│   └─ BlockScan / BlockReduce
│       └─ hardware or logical warps
│           └─ WarpScan / WarpReduce
│               └─ per-thread values/items

```

```

Device API workspace pattern:
call(nullptr, bytes, ...) -> allocate bytes -> call(storage, bytes, ...)

```

6. Common Interview Questions

1. **What is CUB?** Tuned CUDA primitives at thread, warp, block, and device scope. Mistake: calling it only a container library.

2. **How is it different from Thrust?** CUB exposes lower-level CUDA collectives/control; Thrust exposes high-level STL-like algorithms. Mistake: saying one is universally faster.
3. **What is TempStorage?** Scratch storage type required by a specialized collective, commonly allocated in shared memory. Mistake: sharing one instance across independent warps/blocks incorrectly.
4. **Why synchronize before reusing block temporary storage?** Earlier collective threads may still access it; `__syncthreads()` establishes completion before overwrite. Mistake: relying on thread 0 finishing.
5. **What must participate in a block collective?** Threads required by that collective's contract, normally the block, must reach calls consistently. Mistake: placing it in a divergent branch taken by some threads.
6. **Why use BlockLoad?** It can provide coalesced global access and rearrange items into a computation-friendly per-thread layout. Mistake: assuming manual scalar loads are equivalent for every arrangement.
7. **What is the two-call device API pattern?** First query needed workspace with null storage, then allocate and call again. Mistake: querying/allocating every iteration.
8. **How do items per thread affect performance?** More items amortize coordination but increase registers/local state and can reduce occupancy. Mistake: maximizing either occupancy or items blindly.
9. **What is a logical warp?** A compile-time subgroup handled by a warp collective, up to hardware warp size. Mistake: assuming any runtime subgroup size works.
10. **When should CUB replace a custom primitive?** Nearly whenever standard scan/reduce/sort/load semantics fit; custom implementation needs a measured or semantic reason. Mistake: reinventing collectives for learning in production code.

7. Deep-Dive Questions

1. **Why might a CUB-based kernel still be slow?** The primitive can be fast while surrounding loads are uncoalesced, the grid is too small, registers spill, atomics serialize, or multiple kernel phases dominate. Profile source-correlated stalls and the full pipeline.
2. **Can temporary storage be aliased?** Yes when lifetimes do not overlap, but synchronize at the correct scope before reuse. A union can reduce shared-memory footprint for sequential block primitives.
3. **How does specialization help?** Compile-time type, block size, items per thread, and architecture allow unrolling and algorithm selection without runtime branches, at the cost of more compiled variants.
4. **How would you choose warp versus block reduction?** Match the producer/consumer scope. Warp reduction has cheaper coordination but combines fewer values; block reduction handles the whole block and needs broader synchronization/storage.
5. **What can go wrong using device workspace on two streams?** Concurrent algorithms may overwrite shared scratch. Reuse is safe in stream order; otherwise allocate independent storage or establish an event dependency.

8. Comparison Tables

Warp primitive	Block primitive	Device primitive
Up to a hardware warp/logical subgroup	Threads in one block	Entire input/grid
Often shuffle/register oriented	Shared memory + synchronization	Launches kernels, external workspace
Lowest coordination scope	Custom-kernel building block	Complete callable algorithm

Handwritten reduction	CUB reduction
Full control, large correctness surface	Tested, architecture-specialized primitive
Must handle masks, sync, bank patterns	Explicit contract and storage type
Educational or truly specialized	Production default for standard semantics

9. Common Mistakes

- Calling a block collective from only part of the block.
- Reusing `TempStorage` without `__syncthreads()`/appropriate sync.
- Giving multiple warps the same warp scratch instance.
- Hardcoding 32 where logical-warp configuration says otherwise.
- Allocating device-wide workspace inside a hot iteration.
- Tuning template parameters without checking spills and occupancy.
- Assuming use of CUB automatically fixes surrounding memory access.

10. Edge Cases / Special Cases

Partial final tiles need valid-item handling or neutral values. Non-power-of-two logical warp sizes have specific documented behavior. Reduction identity/type promotion, empty device ranges, very large item counts, shared-memory alignment, bank conflicts, and non-associative operators can surprise. Reusing scratch at warp scope needs warp synchronization; at block scope it needs block synchronization.

11. How to Explain in Interview

“CUB is the lower-level CUDA primitives library. I use its warp/block collectives inside custom kernels and its device algorithms for complete scans, reductions, sorts, or selections. The important contracts are participation, temporary-storage lifetime, synchronization, stream ordering, and the resource tradeoff between items per thread and occupancy.”

12. Quick Revision Notes

- Scopes: thread → warp → block → device.
- `TempStorage` is scratch, not output ownership.
- Synchronize before aliasing/reusing scratch.
- Device APIs often query bytes, then execute with caller workspace.
- Items/thread raises amortization and register pressure.
- Interview trap: a collective inside divergent control flow may be invalid.

13. Practice Tasks

1. Implement block sums using `BlockReduce` and finish with `DeviceReduce`.
2. Compact flags using `BlockScan`, including a partial final block.
3. Compare one, two, four, and eight items per thread; inspect register use.
4. Reuse a union of temporary-storage types with correct synchronization.
5. Intentionally share workspace across streams, then fix ordering with events or separate buffers.

14. Final Cheat Sheet

Item	Remember
Core definition	Tuned CUDA collectives from thread to device scope
Why it matters	Reuse correct, architecture-specific parallel primitives
Most asked	Scope, TempStorage , synchronization, workspace
Common comparison	CUB versus Thrust; warp versus block primitives
One-line answer	CUB supplies the optimized parallel gear inside or beneath custom CUDA code.

NCCL

1. Overview

NCCL (pronounced "Nickel") is NVIDIA's topology-aware library for collective and point-to-point communication among GPUs. Core collectives include AllReduce, Broadcast, Reduce, AllGather, ReduceScatter, AllToAll, Gather, and Scatter. It works within a node and across nodes using available interconnects and network transports.

NCCL matters in distributed deep-learning training, multi-GPU analytics, scientific simulations, and model parallelism. Communication frequently becomes the scaling bottleneck after GPU computation is optimized. Interviewers ask about it to test collective semantics, rank/communicator concepts, topology, stream ordering, synchronization, overlap, and deadlock diagnosis.

2. Core Idea

Suppose every GPU computes a gradient. AllReduce sums the gradients and delivers the sum to every GPU. A naive design sends everything to one GPU and broadcasts back, bottlenecking the root. NCCL chooses topology-aware algorithms such as rings or trees and pipelines chunks across NVLink, PCIe, and networks.

```
GPU0: g0 ┌
GPU1: g1 ┤ AllReduce(SUM) -> every GPU receives g0+g1+g2+g3
GPU2: g2 ┤
GPU3: g3 └
```

Each participant has a **rank** in a **communicator**. Every rank issues matching collective calls with compatible count, datatype, reduction/root, and order. Calls enqueue work in CUDA streams. Correct distributed ordering is essential: a mismatch may hang rather than return a neat error.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Rank	Zero-based participant identity inside a communicator.	rank 2 may map to local GPU 0	Rank is not necessarily CUDA

Subtopic	Meaning and why it matters	Example	Common interview angle
			device ID.
Communicator	Context defining ranks/devices participating together.	data-parallel group	Initialization and consistent membership.
AllReduce	Reduce values and distribute result to all ranks.	gradient sum	Equivalent high-level decomposition: ReduceScatter + AllGather.
Broadcast/Reduce	One root sends to all / receives reduction.	parameters/checkpoint stats	Root is a rank, not device ordinal.
AllGather	Every rank contributes a chunk; all receive concatenation.	tensor-parallel activations	Output size grows with rank count.
ReduceScatter	Reduce then give each rank one output shard.	sharded optimizer	Pairs naturally with AllGather.
Topology	NVLink/NVSwitch/PCIe/NIC placement controls paths/bandwidth.	two GPU sockets	Logical rank mapping can affect traffic.
Algorithms/protocols	Ring/tree and protocols trade latency, bandwidth, and topology fit.	small versus large message	No universal winner.
CUDA stream semantics	Communication is enqueued asynchronously and obeys stream dependencies.	compute→event→comm	Correct overlap without host blocking.
Group calls	Aggregate operations or safely issue multi-device calls from one thread.	<code>ncclGroupStart/End</code>	Ordering and enqueue-completion caveats.

4. Real-World Example

In data-parallel training, each GPU processes a mini-batch and produces gradient buckets. As soon as a bucket is ready, an AllReduce is enqueued on a communication stream after an event from the compute stream. Backpropagation continues on later layers while earlier gradients communicate. Buckets must be large enough for bandwidth efficiency but small enough to begin early. At the optimizer boundary, streams synchronize through events so updated weights never consume unfinished gradients.

5. Diagrams / Mental Models

```

Backward compute: [L4 grad][L3 grad][L2 grad][L1 grad]
Communication:    [AR L4] [AR L3] [AR L2] [AR L1]
                  <---- intended overlap ---->

```

Collective	Input per rank	Output per rank
AllReduce	N	N reduced values
AllGather	N	ranks × N concatenated values
ReduceScatter	ranks × N	N reduced shard
Broadcast	N at root	N at every rank
Reduce	N	N at root only

6. Common Interview Questions

1. **What is NCCL?** A topology-aware GPU communication library focused on collectives and point-to-point operations. Mistake: calling it a distributed job scheduler.
2. **What does AllReduce do?** Reduces values across ranks and returns the result to every rank. Mistake: confusing it with Reduce, whose result exists only at root.
3. **What is a rank?** A participant's identity within a communicator, not inherently a global process or local device number. Mistake: using device ordinal as root without mapping.
4. **Why can a ring AllReduce be bandwidth-efficient?** It pipelines chunks and distributes transfer load across links; latency grows with more steps. Mistake: saying every rank sends all data directly to all others.
5. **Why use a tree?** It can reduce step latency for small messages/rank counts, while ring-like schemes often approach link bandwidth for large messages. Mistake: choosing only by rank count.
6. **Are NCCL calls synchronous?** They enqueue asynchronous device work on a CUDA stream after the operation is posted. Completion uses normal CUDA events/synchronization. Mistake: assuming API return means collective complete.
7. **What causes NCCL hangs?** Mismatched operation order/count/type/root, missing rank, communicator errors, process failure, or bad stream/group coordination. Mistake: debugging only network bandwidth.
8. **How do you overlap communication and compute?** Use separate streams plus events/dependencies, split data into sensible buckets, and ensure independent resources. Mistake: separate streams with no dependency correctness.
9. **Why does topology matter?** GPUs may connect through NVLink/NVSwitch, PCIe switches/CPU, or NICs with different bandwidth/latency/contention. Mistake: assuming all GPU pairs are equal.
10. **Why group calls?** Manage multiple devices from one thread, aggregate launches, or combine point-to-point operations. Mistake: synchronizing a grouped stream before `nccclGroupEnd` has posted work.

7. Deep-Dive Questions

1. **How is AllReduce related to ReduceScatter and AllGather?** ReduceScatter produces disjoint reduced shards; AllGather exchanges those shards so every rank obtains the full reduced tensor. Efficient AllReduce implementations commonly reflect this structure.

2. **Why might overlap fail even with two streams?** Kernels may contend for SMs/memory/network copy resources, dependencies may serialize, communication may start too late, messages may be too small, or framework/NCCL ordering may create synchronization. Verify with a timeline.
3. **How do bucket sizes affect training?** Small buckets start earlier but pay more launch/protocol latency; large buckets use bandwidth efficiently but delay communication and reduce overlap. Tune from measured exposed communication time.
4. **How would you diagnose multi-node slowdown?** Compare single-GPU, intra-node, and inter-node baselines; inspect topology/rank placement, message-size curves, NIC/GPU affinity, transport selection, link errors/contention, and whether computation leaves communication exposed.
5. **Why must collective order match?** Ranks cooperatively execute one protocol. If rank A expects AllReduce while rank B expects Broadcast or another communicator sequence, their sends/receives do not pair, causing undefined behavior such as hangs or corruption.

8. Comparison Tables

NCCL	MPI
GPU-collective focused, CUDA-stream integrated	General distributed message-passing standard
Topology-aware GPU data paths	Broad CPU/network ecosystem
Device buffers and kernels central	CPU and many accelerator integrations
Often used beneath DL frameworks	Used for general HPC control/data exchange
Ring	Tree
Strong large-message bandwidth utilization	Fewer logical steps/low-latency potential
More steps as ranks increase	Link/root-level contention can matter
Pipeline-friendly	Often useful for smaller messages
Actual choice is topology/protocol dependent	Actual choice is topology/protocol dependent

9. Common Mistakes

- Issuing different collective order or parameters across ranks.
- Confusing rank ID with CUDA device ordinal.
- Measuring the host enqueue call rather than stream completion.
- Assuming separate streams guarantee overlap.
- Creating extremely small gradient buckets.
- Ignoring GPU/NIC/NUMA topology and rank placement.
- Treating a communication hang as necessarily a network failure.

10. Edge Cases / Special Cases

One-rank communicators, zero counts, in-place collective rules, multiple communicators, multiple ranks mapped to one GPU, process failure, asynchronous errors, graph capture, and mixing streams within a group require documented handling. All participants must agree on ordering and compatible metadata. Some grouped calls are not fully enqueued until the outermost `ncclGroupEnd`, so premature stream synchronization is unsafe.

11. How to Explain in Interview

"NCCL is NVIDIA's topology-aware GPU collective communication library. Ranks in a communicator issue matching collectives such as AllReduce on CUDA streams. Performance depends on message size, topology, algorithm/protocol, rank placement, and overlap; correctness depends on identical collective ordering and stream dependencies across ranks."

12. Quick Revision Notes

- AllReduce = reduce result delivered to all ranks.
- Rank belongs to a communicator; it is not automatically device ID.
- Calls are stream-ordered and asynchronous with respect to completion.
- Ring favors pipelined bandwidth; trees can favor latency.
- Overlap must be proven in a timeline.
- Interview trap: mismatched collectives often hang.

13. Practice Tasks

1. Trace a four-rank ring AllReduce by chunks on paper.
2. Run AllReduce correctness tests for sum/min/max across message sizes.
3. Plot algorithm bandwidth and latency for intra-node and inter-node cases.
4. Overlap a synthetic compute kernel with AllReduce using events; inspect the timeline.
5. Deliberately mismatch call order in a controlled test and practice diagnosing logs/timeouts.

14. Final Cheat Sheet

Item	Remember
Core definition	Topology-aware multi-GPU collective communication library
Why it matters	Communication determines distributed scaling
Most asked	AllReduce, ranks, ring/tree, streams, hangs, overlap
Common comparison	NCCL versus MPI; ring versus tree
One-line answer	NCCL moves and reduces GPU data efficiently when every rank follows the same protocol.

Why Is My CUDA Kernel Slow?

1. Overview

"Why is my kernel slow?" is not answered by one metric. A kernel can be slow because the GPU was not doing it for long enough to amortize launch cost, because there is too little parallel work, or because its limiting resource is memory bandwidth, memory latency, instruction throughput, dependencies, divergence, atomics, synchronization, registers, shared memory, or communication. It may also be fast while the **application** is slow due to transfers, allocations, synchronization, or excessive launches.

This matters in every GPU system. Interviewers ask it to test whether a candidate uses a disciplined performance model rather than reciting “increase occupancy” or “use shared memory.” A good answer begins with measurement, identifies the bottleneck, changes one limiting cause, and validates both speed and correctness.

2. Core Idea

Treat the GPU like a factory:

- **Demand:** how much useful work exists?
- **Supply:** how much compute and memory capability can the GPU provide?
- **Flow:** are enough warps ready, and can data/instructions reach them?
- **Overhead:** how much time is spent starting, waiting, copying, and coordinating?

The roofline model provides a first classification:

$$\text{arithmetic intensity} = \frac{\text{useful operations}}{\text{bytes transferred from limiting memory level}}$$

$$\text{attainable performance} \leq \min(\text{peak compute}, \text{bandwidth} \times \text{arithmetic intensity})$$

Example: vector addition performs roughly one add while reading two inputs and writing one output. Its low arithmetic intensity makes it bandwidth-bound; adding more arithmetic units or chasing 100% occupancy will not exceed the memory roof. Matrix multiplication reuses tiles many times, so a sufficiently large GEMM can become compute-bound.

Step-by-step diagnosis:

1. Correct timing and application timeline
2. Find the real hot kernel / exposed gap
3. Check problem size and launch overhead
4. Classify compute-, bandwidth-, or latency-limited
5. Inspect limiting stalls/resources/source lines
6. Make one hypothesis-driven change
7. Re-measure performance and correctness

3. Important Subtopics

Subtopic	What it means and why it matters	Evidence/example	Common interview angle
Correct timing	GPU launches are asynchronous; CPU wall time around launch is enqueue time.	CUDA events or profiler duration	Synchronize the stop event, not every operation.
Timeline	Host/device copies, gaps, serialization, and tiny launches may dominate outside one kernel.	Nsight Systems	Optimize exposed critical-path time.

Subtopic	What it means and why it matters	Evidence/example	Common interview angle
Arithmetic intensity	Useful operations per byte moved.	SAXPY low, tiled GEMM high	Roofline classification.
Achieved bandwidth	Actual bytes/time compared with sustainable device bandwidth.	coalesced copy baseline	Low percentage may mean latency/transactions, not "compute-bound."
Compute throughput	Utilization of relevant pipelines: FP32, Tensor Core, integer, special functions.	instruction mix and utilization	Peak spec must match instruction/data type.
Occupancy	Active warps relative to architectural maximum. It helps hide latency but is not performance itself.	limited by registers/shared memory/blocks	Higher occupancy can hurt through spills.
Parallelism	Enough blocks/warps to fill SMs and provide ready work.	tiny grid or long tail	One block per SM is not automatically enough.
Memory coalescing	Warp addresses should map to few useful transactions.	contiguous lane access	AoS/stride/scatter transaction waste.
Local-memory spills	Excess registers place values in per-thread local memory backed by device memory/cache.	compiler report, local load/store	Reducing registers may trade occupancy and recomputation.
Shared memory	Reuses data and enables cooperation but costs occupancy and synchronization.	tiled stencil/GEMM	It helps only when reuse exceeds overhead.
Bank conflicts	Shared-memory lane accesses serialize when mapping conflicts, except broadcasts/supported patterns.	conflict metrics	Padding/transposition fixes patterns.
Divergence	Lanes in a warp take different paths or loop counts.	low active lanes/branch efficiency	Branch presence is not divergence.
Latency dependencies	Warps wait on loads or dependent instruction chains; insufficient	long scoreboard/dependency stalls	More independent work, locality, or warps.

Subtopic	What it means and why it matters	Evidence/example	Common interview angle
	independent work exposes latency.		
Atomics/contention	Threads serialize on hot addresses or cache lines.	histogram hot bins	Privatize/aggregate if the extra work pays.
Synchronization	Barriers, events, and host sync leave workers idle and block overlap.	barrier stalls, timeline gaps	Remove only if correctness permits.
Launch configuration	Block size affects warps, resource allocation, tail effects, and mapping.	occupancy calculator plus benchmark	Multiples of warp size are a start, not a proof.

4. Real-World Example

Consider a histogram kernel that is ten times slower on skewed input than uniform input. The grid is large and memory reads are coalesced. Nsight Compute shows high atomic serialization, and source correlation identifies `atomicAdd(&global_bins[key], 1)` as the hot instruction. Skew sends most threads to one bin.

A justified solution is hierarchical aggregation: each block accumulates a private shared-memory histogram, then merges bins into global memory. This replaces many contended global atomics with cheaper local aggregation and fewer global atomics. But if the histogram has millions of bins, a private copy is too large; alternatives include warp aggregation, partitioning, sorting/reduce-by-key, or accepting atomics. The correct change depends on bin count and distribution, not merely the word “atomic.”

5. Diagrams / Mental Models

```

Application slow?
├ GPU has large idle gaps → CPU launch/dependency/I/O/allocation problem
├ many tiny kernels      → launch-bound; batch/fuse/graphs
├ one dominant kernel
│   ├── near bandwidth roof → reduce bytes, improve reuse/compression/fusion
│   ├── near compute roof   → reduce instructions/use faster suitable math
│   └ near neither
│       ├── too little work  → larger batch/grid or different mapping
│       ├── low ready warps  → registers/shared memory/block limits
│       ├── memory stalls    → coalescing, locality, ILP, latency hiding
│       ├── divergence       → group coherent work/reformulate if worthwhile
│       └ serialization      → atomics/barriers/dependencies
└ transfers dominate      → keep data resident, batch, pin/overlap if valid

```

Symptom	Do not conclude immediately	Check next
Low occupancy	“That is the bug”	Eligible warps, stalls, spills, achieved roofs

Symptom	Do not conclude immediately	Check next
Low bandwidth	"It is compute-bound"	Irregular transactions, latency, small grid
Low compute utilization	"Need more math"	Memory/latency/serialization/launch limits
High cache hit rate	"Memory is fine"	Requested versus transferred bytes and latency
Many branches	"Divergence"	Outcomes within each warp and path cost

6. Common Interview Questions

- How do you start diagnosing a slow CUDA kernel?** Establish correct timing and correctness, inspect the application timeline, find the hot kernel, then use Nsight Compute/roofline and stall evidence. Expected: measurement before code changes. Mistake: proposing shared memory immediately.
- Does low occupancy always mean poor performance?** No. Enough warps may hide latency, and compute- or bandwidth-saturated kernels need not reach maximum occupancy. Very low occupancy can hurt, but it is evidence, not a root cause. Mistake: targeting 100% at any cost.
- Does high occupancy guarantee speed?** No; warps can all wait on uncoalesced memory, atomics, barriers, or dependencies. Mistake: confusing residents with productive instructions.
- How do you know a kernel is memory-bandwidth-bound?** Estimate bytes and arithmetic intensity, measure achieved bandwidth against a sustainable baseline, and confirm memory throughput/stalls. Mistake: declaring it from low FLOP rate alone.
- How do you know it is compute-bound?** Relevant compute pipelines approach their attainable throughput while memory is not the limiting roof; instruction mix/source confirms useful compute. Mistake: comparing integer/SFU code with FP32 peak.
- What is coalescing?** Warp memory requests combine into the minimum practical aligned transactions when lanes access nearby addresses. Mistake: saying all consecutive per-thread code is coalesced regardless of addresses.
- What are register spills?** Live values that do not fit allocated registers use local memory, potentially adding cache/device traffic. Expected: inspect compiler/profiler evidence. Mistake: forcing fewer registers without checking new spills/recomputation.
- Why can a small kernel be slow?** Launch latency and insufficient blocks/warps dominate; peak throughput assumes enough work. Expected: batching/fusion or CPU crossover. Mistake: optimizing instruction-level details first.
- When does shared memory help?** When it enables reuse, coalescing, or cooperation enough to exceed loads, barriers, bank conflicts, and occupancy cost. Mistake: copying once-used data through shared memory.
- How do you optimize atomics?** First measure contention and address distribution; aggregate per warp/block, privatize, partition, or change algorithm when reduced contention repays extra storage/work. Mistake: claiming atomics are always slow on modern GPUs.

7. Deep-Dive Questions

- A kernel reaches only 30% of peak bandwidth and 10% of peak compute. What now?** It may be latency/serialization/underutilization-bound. Check grid size, eligible versus active warps, memory

transaction efficiency, cache misses, long-scoreboard stalls, dependent instruction chains, divergence, atomics, and tail effects. Low percentages of both roofs do not identify a resource by themselves.

2. **Why can reducing register count make a kernel slower despite raising occupancy?** The compiler may spill to local memory or shorten value lifetimes through extra instructions. Additional resident warps may not compensate for added traffic/dependencies. Sweep launch bounds/register limits only with measurements.
3. **Why might a fused kernel underperform two kernels?** Fusion can raise registers/shared memory, lower occupancy, reduce scheduling flexibility, repeat work, or combine incompatible launch shapes. It wins only if saved launches/intermediate traffic exceed those costs.
4. **How do tail effects reduce utilization?** Blocks/warps have unequal durations; near the end of a wave, a few long-running blocks occupy SMs while others become idle. Balance work, increase grid granularity, or use persistent/dynamic scheduling when overhead is justified.
5. **How do you distinguish requested bytes from transferred bytes?** Source semantics count requested useful bytes; hardware sectors/transactions count actual movement. Their ratio reveals wasted transactions from stride, misalignment, partial warps, or cache-line overfetch. Both are needed for meaningful bandwidth analysis.

8. Comparison Tables

Compute-bound	Bandwidth-bound	Latency-bound
Relevant execution pipeline saturated	Memory throughput near sustainable roof	Neither roof reached; warps frequently wait
Reduce operations or use suitable lower precision/instructions	Reduce bytes, reuse/fuse, coalesce	Improve locality, ILP, ready warps, parallelism
More occupancy often has limited value	More FLOP units do not help	Occupancy may help if resources allow
Nsight Systems	Nsight Compute	
Whole application timeline	Detailed individual-kernel analysis	
CPU threads, launches, copies, streams, gaps	Throughput, stalls, occupancy, source/SASS metrics	
Answers “where is time exposed?”	Answers “why does this kernel behave so?”	
Use first for application bottlenecks	Use next for the selected kernel	
Optimization	Helps when	Can hurt through
More occupancy	Latency is exposed and more warps become eligible	Register restriction/spills
Shared-memory tiling	Data is reused or access regularized	Barriers, banks, capacity
Fusion	Intermediates/launches dominate	Register pressure, poor launch shape
Fast/reduced precision	Error tolerance permits and hardware accelerates it	Accuracy/range loss, conversions

Optimization	Helps when	Can hurt through
Batching	Work items are small and independent	Added latency or padding

9. Common Mistakes

- Timing asynchronous launches with a CPU timer and no completion event.
- Optimizing the longest kernel when it overlaps and is not on the critical path.
- Treating occupancy as a score to maximize.
- Using peak theoretical bandwidth/FLOPS without a sustainable baseline or correct instruction type.
- Adding shared memory without reuse.
- Removing all branches although warp outcomes are coherent.
- Forcing register limits and creating spills.
- Ignoring transfers, allocation, initialization, warm-up, and launch count.
- Changing several things at once, making causality unknowable.
- Reporting speedup without validating output and representative inputs.

10. Edge Cases / Special Cases

First launches include context/module initialization; debug builds and profiler instrumentation perturb timing; GPU clocks, thermals, power limits, and concurrent processes add variance. Unified Memory page faults can dominate only some iterations. Small edge tiles may diverge but be negligible. Clock measurements inside kernels require careful interpretation. CUDA events measure stream elapsed GPU time, while end-to-end latency may require wall time. A kernel can become faster but make the pipeline slower by preventing overlap or increasing a downstream conversion.

11. How to Explain in Interview

"I do not diagnose a slow kernel from occupancy alone. I first verify timing and use a system timeline to find exposed critical-path work. For the hot kernel, I estimate arithmetic intensity and compare achieved compute and bandwidth with attainable roofs. If neither is saturated, I inspect ready warps, memory transactions and stalls, divergence, atomics, dependencies, register/shared-memory limits, and grid size. I change the measured bottleneck, then re-profile and revalidate correctness."

12. Quick Revision Notes

- Measure: warm up, use CUDA events/profilers, validate results.
- Systems timeline first; kernel metrics second.
- Roofline: $\min(\text{compute peak}, \text{bandwidth} \times \text{arithmetic intensity})$.
- Occupancy hides latency; it is not utilization or speed.
- Low compute + low bandwidth often means latency/serialization/too little work.
- Coalescing concerns addresses per warp, not source-code neatness.
- Shared memory needs reuse/cooperation to pay.
- Optimize the critical path, not the prettiest metric.

13. Practice Tasks

1. Time vector addition incorrectly with CPU timestamps, then correctly with CUDA events; explain the difference.

2. Write coalesced and stride-32 access kernels and compare transactions/bandwidth.
3. Build naive and tiled matrix multiplication; calculate arithmetic intensity and compare with cuBLAS.
4. Vary block size and record registers, active warps, spills, and runtime; show why occupancy alone fails.
5. Create uniform and divergent branch patterns with equal global branch counts; compare warp behavior.
6. Benchmark a contended global histogram and a block-private version on uniform and skewed inputs.
7. Use Nsight Systems to find hidden synchronization or many tiny kernels in a real pipeline.
8. Produce a one-page report: hypothesis, evidence, change, correctness check, before/after result.

14. Final Cheat Sheet

Item	Remember
Core definition	Kernel performance is constrained by its active limiting resource and overhead
Why it matters	GPU tuning must follow evidence, not folklore
Most asked	Memory vs compute bound, occupancy, coalescing, divergence, spills
Common comparison	Nsight Systems versus Compute; compute/bandwidth/latency bound
One-line interview answer	Measure the critical path, classify the bottleneck, inspect its evidence, change one cause, and re-measure.

Cross-Library Decision Guide

Workload shape	Prefer	Why	Watch for
Standard dense algebra	cuBLAS	Tuned BLAS and Tensor Core paths	layout, leading dimensions, tiny shapes
DNN operator/fusion graph	cuDNN	Shape-aware engines and fusion	plan warm-up, workspace, determinism
Spectral transform	cuFFT	Planned multidimensional/batched FFT	normalization, layout, plan reuse
Sparse matrix operation	cuSPARSE	Format-aware sparse algorithms	density, imbalance, indices
Complete standard C++ parallel operation	Thrust	Most expressive high-level API	launches, temporaries, host sync
Primitive inside custom CUDA work	CUB	Warp/block/device collectives	participation, scratch, sync
Communication across GPUs	NCCL	Topology-aware collectives	matching order, topology, overlap

Workload shape	Prefer	Why	Watch for
Truly unsupported fused/specialized operation	Custom kernel, often using CUB	Only path with required semantics	maintenance and architecture tuning

Official References

- [cuBLAS documentation](#)
- [cuDNN documentation](#)
- [cuFFT documentation](#)
- [cuSPARSE documentation](#)
- [Thrust documentation](#)
- [CUB documentation](#)
- [NCCL documentation](#)
- [CUDA C++ Best Practices Guide](#)